



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目	面向低时延大语言模型推理的 Megakernel		
	依赖感知物理分区与 PDL 跨边界执行研究		
学生姓名	郝朝阳	学号	待填写
指导教师	待填写	职称	待填写
学位类别	工学硕士		
学科专业	计算机科学与技术		
研究方向	AI 编译器与 GPU 高性能计算		
培养单位	处理器芯片全国重点实验室		
填表日期	2026 年 07 月		

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 研究背景	1
1.2 选题意义	4
二 国内外本学科领域的发展现状与趋势	5
2.1 从图级融合到 kernel orchestration	5
2.2 Whole-model 与 persistent Megakernel	6
2.3 动态 task/event 与自动执行计划生成	7
2.4 逻辑 Megakernel 的多 grid 物理映射	7
2.5 CUDA Graph 与 Programmatic Dependent Launch	8
2.6 现状归纳与研究机会	9
三 课题主要研究内容、预期目标	9
3.1 问题定义与总体目标	9
3.2 研究问题	11
3.3 可证伪研究假设	12
3.4 主要研究内容	13
3.5 预期目标、创新点与成果	14
四 拟采用的研究方法、技术路线、实验方案及其可行性分析 ..	15
4.1 问题建模与分析对象	15
4.2 四路因果对照与关键路径定位	16
4.3 边界特征与依赖实现层级选择	18
4.4 PDL 跨边界变换与收益模型	18
4.5 实验方案	19
4.6 可行性分析与风险控制	21
五 已有科研基础与所需的科研条件	22

5.1 真实 Megakernel 的实验基础	22
5.2 SM、SASS 与关键路径分析基础	22
5.3 PDL 与跨代码库实现基础	24
5.4 软硬件与实验条件	26
六 研究工作计划与进度安排	26

一 选题的背景及意义

1.1 研究背景

大语言模型推理将一个 Transformer 层映射为归一化、线性变换、位置编码、Attention、激活函数和残差连接等 GPU 计算阶段。预填充阶段通过较大的 token 维度形成并行性；解码阶段每轮仅产生少量新 token，且后一轮依赖前一轮生成的 KV Cache。随着 batch size 和单步计算规模下降，kernel 启动、跨 kernel 同步、中间张量物化和设备侧调度在关键路径中的占比上升。FlashAttention 通过重新组织 Attention 的数据流减少 HBM 访问，TVM、DNNFusion 和 Welder 等编译系统通过算子融合扩大片上数据复用范围，这些工作共同表明：低时延推理的性能取决于算子内部实现与算子之间执行边界的协同设计[Dao 等 \(2022\)](#); [Chen 等 \(2018\)](#); [Niu 等 \(2021\)](#); [Shi 等 \(2023\)](#)。

近期研究进一步将融合范围从局部算子扩展到 Transformer 层乃至完整解码主体。FlashFormer 将 whole-model kernel 用于低 batch 推理，MegaQwen 将 Qwen3-0.6B 的主要解码计算组织在 cooperative kernel 内，Hazy Research Megakernels 和 MPK 将多个算子编译为设备端任务图，Event Tensor 则扩展了动态 shape 与动态任务调度[Nrusimha 等 \(2025\)](#); [Infatoshii \(2026a\)](#); [Hazy Research \(2025a,b\)](#); [Cheng 等 \(2026\)](#); [Jin 等 \(2026\)](#)。这类执行方式统称为 Megakernel：CPU 只发起一次或少量物理 CUDA launch，一组 persistent 或 cooperative workers 在 GPU 内部连续执行多个算子任务，并通过 queue、event、counter 或 grid-wide synchronization 表达跨算子依赖。

图 1 给出普通多 kernel 执行与任务图式 Megakernel 的差异。普通执行使用 kernel completion 表达相邻阶段之间的全局顺序；Megakernel 将该顺序下沉到设备端，使不同 SM 能够沿任务图独立推进。当后继算子只依赖前驱的一部分输出时，设备端事件可以把完整的 grid barrier 细化为 per-tile、per-head 或 per-column 的流水交接。该机制同时减少 launch 并扩大跨算子重叠，是 Megakernel 获得低时延收益的核心来源。

Megakernel 消除物理边界后，跨算子依赖仍需一个可执行的实现。设备端调度器需要决定后继任务何时进入、由谁判断全局 ready、等待中的 SM 如何让出执行资源、ready 状态如何传播，以及前驱写入何时对后继可见。细粒度依赖能

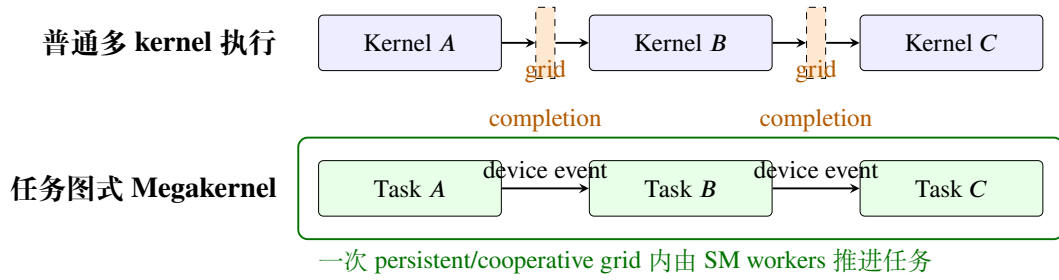


图 1 普通多 kernel 执行与任务图式 Megakernel。物理 grid 边界提供全局完成顺序；Megakernel 使用设备端事件表达相同逻辑依赖，并在依赖允许时形成细粒度流水。

够用这些软件协议换取真实的 dependent computation overlap；全量依赖则呈现不同结构：后继的主要计算共享同一个 all-or-nothing ready 条件，在最后一块前驱输出到达以前，后继仅能执行独立准备或等待。

这一差异产生了本课题的核心研究机会。设计算图中相邻算子阶段之间形成一个依赖切口。当后继可以按 tile 或 head 消费前驱的局部输出时，同一 grid 内的事件同步保留了细粒度流水；当后继的主要任务都等待前驱的全量结果时，SM-local task admission 与 global readiness 之间出现粒度差异。物理 grid completion 恰好以全局粒度表达后一类依赖，并同时结束前驱阶段的 persistent execution state。此时，减少一次 launch 与减少一次 single-grid 软件交接成为两个方向相反的性能项，最优实现由依赖拓扑、投放时机、阶段长度、数据局部性和边界成本共同决定。

前期实验选取 Hazy Research Megakernels 的真实算子阶段切口作为研究对象。该切口由 PartialAttention 阶段指向 O_ProjResidual 阶段。该配置由 8 个 PartialAttention tasks 产生 32 个 query heads；每个 O-projection task 计算 16 个输出行，同时读取完整的 attn_out[2048]。因此，128 个 O-projection tasks 都等待同一个值为 32 的全局 ready counter。Hazy VM 依据运行时 SMID 选择 worker queue：queues 0–7 执行长 K/V scan，queues 8–131 在相邻 slot 中进入 O-projection。131,072-token context（下文简称 131K）的逐 SM 诊断记录显示，124 个候选 queues 在三次重复中的 372/372 次观测均在全量 ready 前进入依赖等待；原始路径中 128 个 O tasks 的 384/384 次 first TMA 观测发生在 ready 之前。

在保持 task 实现、SM queue 映射、输入输出和 33,792 个 instruction bytes 一致的条件下，将这条边实现为两个同 stream 的物理 grid，在 131K context 上稳定节省 45.42–46.64 μ s，对应 1.409%–1.443%；120K 配套 Nsight Systems 时间线中

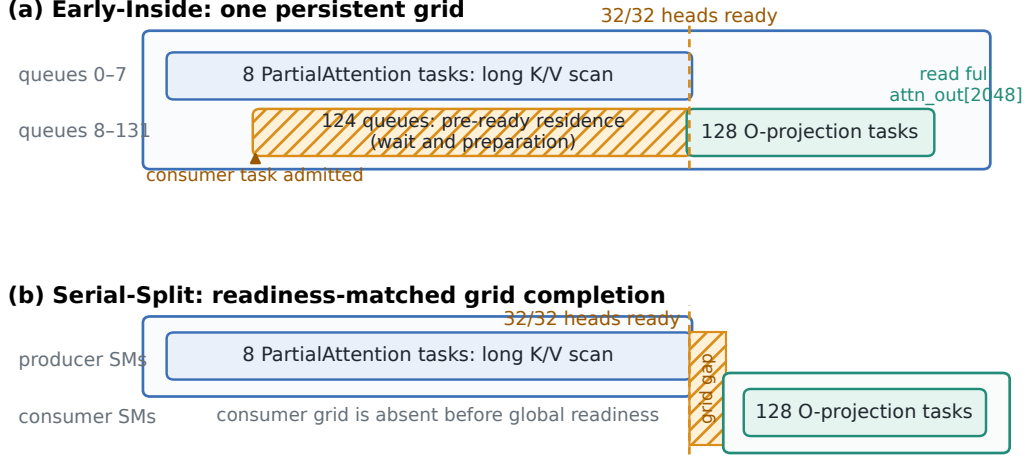


图 2 Hazy PartialAttention→O_ProjResidual 的 SM-时间执行结构。Early-Inside 中，queues 8-131 在 32 个 query heads 全量就绪前进入 consumer 路径，形成 pre-ready residence interval；Serial-Split 由 producer grid completion 统一放行 consumer grid。两种方案保持 task code、queue mapping、task/noop slots 和 instruction bytes 一致。橙色区间对应后续 admission-gated 对照与设备端时间戳所分解的关键阶段。

的可见 inter-kernel gap 中位数为 $1.056 \mu s$ 。同一计算图的三个相邻边呈现相反方向。表 1 将实测方向与边界结构并列给出。这组正负样本将物理边界从抽象设想转化为可重复测量的执行计划选择。

进一步的因子实验构造了 first-TMA 时序与 polling backoff 的 2×2 对照，backoff 分别取 1 ns 和 1024 ns。两个 H800、三个独立进程、四个实验 cell 共 12 个 paired median 中，split 收益保持在 $44.832-47.776 \mu s$ 。first TMA 被移动到 ready 后，polling load 也显著下降，split 差距仍然存在。这两个干预分别排除了 ready 前 weight TMA 和高频 polling load 数量作为 $45-47 \mu s$ 差距的单一主因。下一阶段优先检验 task admission 与跨阶段交接：损失可能出现在 producer 最后 ready 之前，也可能出现在 ready 后的 controller、instruction ring、semaphore、barrier 与 grid drain 阶段，两部分可同时进入关键路径。

Hopper 的 Programmatic Dependent Launch (PDL) 为选定物理边界提供了第二个机会。PDL 允许同一 stream 中的后继 grid 在前驱满足 programmatic completion 条件后取得调度资格；后继可以先执行地址准备、descriptor 设置和独立权重预取，并在首次读取前驱输出前完成依赖同步 [NVIDIA \(2026d,b\)](#)。前期实验在一张物理 H800 上使用修改后的匹配 binary，对同一 Hazy 正边比较 ordinary split 与 split+PDL，测得 PDL 进一步节省 $2.4-3.8 \mu s$ 。因此，本课题形成一条连续的优化

表 1 Hazy 同一计算图上相邻边的物理分区结果

相邻算子边	readiness 与 producer 特征	Workload 与独立单元	物理分区结果
PartialAttention → O-projection	32 heads 全量就绪; producer 随 context 增长; 128 个 consumer 共享同一 full-ready 条件	131K; 3 个进程, 各 100 对	131K 节省 45.42–46.64 μ s (1.409%–1.443%)
QKV/RoPE → PartialAttention	K/V head 与四个 Q heads 局部就绪; grid 内可按 head 开始扫描历史 KV	32K; 2 个进程, 各 200 对	32K 增加 46.77–47.10 μ s (5.460%–5.488%)
O-projection → UpGate	producer 约 8 μ s; 当前 Python/pybind 提交路径暴露边界 gap	32K; 2 个进程, 各 200 对	增加 49.18–51.33 μ s (53.993%–55.966%)
UpGate → DownProj	4 个 reduction-column counters 支持按列推进	32K; 1 个进程, 200 对	增加 30.06 μ s (31.078%)

路径：首先选择依赖最合适的实现层级，再利用 PDL 压缩已选物理边界的交接成本。

基于上述观察，本课题研究**依赖感知的 Megakernel 物理分区与 PDL 跨边界优化**。核心对象是一个已经生成的 Megakernel task graph 及其相邻依赖边；核心决策是在同一 grid 内的细粒度事件、投放前的内部 readiness gate 与跨 grid completion 之间选择依赖实现，并在物理边界上联合优化 PDL trigger 和 consumer 独立前缀。该问题连接了图级融合、设备端调度和硬件依赖执行三个层次，为低时延 LLM 推理提供了新的执行计划空间。

1.2 选题意义

本课题首先具有执行模型层面的研究意义。Megakernel 将 kernel launch 从主机侧移除后，跨算子依赖由设备端软件协议承载。依赖 ready 粒度、task admission 粒度与物理执行粒度共同决定协议成本和跨算子流水收益。将三者放入同一个分析框架，可以解释同一计算图中相邻边为何出现不同的最优实现，并把 Megakernel 的性能分析从“融合范围”推进到“依赖应由哪一级机制实现”。

其次，本课题为编译器提供了新的物理映射决策。现有融合系统通常围绕中间数据复用、launch 消除和资源约束生成融合区域；任务图式 Megakernel 进一步引入了 worker queue、task admission、全局 counter 和持久化 grid 生命周期。物理分区把这些运行时状态纳入图分区目标，使编译器可以在保持逻辑 task graph 的同时，为不同依赖选择细粒度 event、内部 global gate 或 grid completion。该抽象能够统一表示保留流水、推迟后继投放和隔离阶段状态三种执行效果。

再次，本课题拓展了 PDL 的应用位置。PDL 现有示例主要围绕已经存在的 producer-consumer kernels，研究 trigger 与独立权重预取 NVIDIA (2026d,b)。物理分区将“边界选择”和“边界交接”连接起来：前一阶段确定 grid completion 是否与依赖拓扑匹配，后一阶段从 consumer 中提取合法独立前缀，并搜索 trigger、预取量和存储层级。由此得到的优化空间同时包含 single-grid 软件同步成本、物理边界成本和跨边界重叠收益。

最后，本课题具有直接的工程价值。低 batch decode 的单步时延会沿 token 序列累积，几十微秒的局部关键路径差值能够影响 TPOT 与交互响应。H800 上的前期结果已经给出一条稳定的真实正边和三条同图负边，为边界选择提供了可复现实验基础。后续将把边界接入 decoder layer 和完整 decode step，并在第二类真实计算图上验证特征迁移，从而形成可嵌入 Megakernel 编译器或推理运行时的执行计划选择方法。

二 国内外本学科领域的发展现状与趋势

2.1 从图级融合到 kernel orchestration

GPU 算子融合和优化单位经历了从算子节点到 tile 与 primitive 的持续细化。TVM 在张量表达和计算图层执行算子融合与跨算子优化，DNNFusion 根据算子关系和数据流属性生成融合计划，Welder 进一步以 tile graph 描述生产者与消费者之间的数据复用，从而联合调度算子内与算子间访存 Chen 等 (2018); Niu 等 (2021); Shi 等 (2023)。FlashAttention 则从算法层重排 Attention 的分块与归约，使中间 attention matrix 保持在片上存储层级，展示了数据流重构与 kernel 边界设计的协同收益 Dao 等 (2022); Dao (2024); Shah 等 (2024)。

Korch 将该方向推进到 kernel orchestration。其方法先把张量算子拆成基础

代数 primitives，再用二元线性规划选择覆盖整个 primitive graph 的 kernel 集合；优化器由此能够在算子内部切分计算，并跨越原始算子边界重新组合 kernelHu 等 (2024)。ClusterFusion 使用 cluster-level collective primitive 扩大协作范围，使跨 thread block 的数据交换与同步进入融合搜索空间Luo 等 (2025)。这些系统共同建立了一个基本结论：逻辑计算图到 GPU kernel 的映射是一个组合优化问题，收益同时取决于 launch 数量、中间数据物化、片上复用、同步和资源占用。

上述方法主要在代码生成阶段决定融合区域和 kernel 组成。Megakernel 将同一问题继续推进到设备端执行阶段：多个算子已经进入同一逻辑任务图之后，依赖由何种运行时机制承载、任务以何种粒度投放、逻辑任务图映射为几个物理 grid，成为新的物理执行计划变量。

2.2 Whole-model 与 persistent Megakernel

Whole-model kernel 以固定 cooperative grid 执行一层或多层 Transformer 阶段。FlashFormer 面向低 batch 推理，把 whole-model 数据流映射到单个 kernel，并通过跨阶段流水缓解权重搬运与计算之间的空隙Nrusimha 等 (2025)。MegaQwen 的固定提交 0bc1fded 将 Qwen3-0.6B 的 RMSNorm、QKV projection、RoPE、Attention、O projection 和 MLP 等主要 decode 计算组织在 cooperative CUDA 路径中；其公开结果同时记录了 context length 增长时吞吐下降，以及大量 grid.sync() 对短序列时延的影响Infatashi (2026a,b)。这类静态方案在编译期确定工作分工和同步位置，执行时以 grid-wide barrier 推进固定阶段序列。

Persistent Megakernel 使用常驻 workers 执行设备端 task graph。Hazy Research Megakernels 将算子任务编译为各 SM 的指令队列。worker 根据 event、counter 和 barrier 推进跨算子依赖Hazy Research (2025a,b)。MPK 在 OSDI'26 中提出 SM-level graph 表示，将张量程序降低为细粒度 tasks，并由去中心化的设备内运行时在 persistent megakernel 中调度；论文报告的端到端时延改进最高达到 1.7×Cheng 等 (2026)。与 cooperative whole-model kernel 相比，task-graph runtime 能够在依赖允许时让不同 SM 独立推进，从而把整 grid 同步细化为 task 或 tile 级事件。

这两类实现采用不同的调度结构，却承担相同的系统职责：一次物理 grid 需要在多个阶段间维持任务发现、ready 判断、数据可见性和 worker 前向进展。其关键收益来自消除 launch 与中间物化，并把局部 readiness 转化为跨算子流水；

其关键成本来自设备端调度、软件同步和不同阶段共享同一 persistent execution state。

2.3 动态 task/event 与自动执行计划生成

真实推理中的 sequence length、batch size 和 MoE routing 会改变 task 数量及依赖关系。Event Tensor 把事件状态提升为 tensor abstraction，用静态和动态 event update/trigger 表达 shape-dependent 与 data-dependent task graph，并由编译器生成 persistent kernel 的静态或动态调度Jin 等 (2026)。Ada-MK 针对固定部署配置执行细粒度 DAG 离线搜索，把运行时分支前移到编译期，同时用异构引擎分别承载 prefill 与 decodeDong 等 (2026)。两项工作从不同方向说明，Megakernel 执行计划由 workload shape、依赖结构和硬件资源共同确定。

自动化工具也开始进入 Megakernel 代码生成。AutoMegaKernel 以 schedule IR validator 检查 agent 生成计划的 race、deadlock 和资源约束，并从统一代码生成面向多代 NVIDIA GPU 的 persistent cooperative kernelJaber 等 (2026)。这一进展使执行计划搜索具备更大的候选空间，也提高了对可验证依赖表示和可复现实验接口的要求。对物理分区而言，边界决策需要与 task schedule 一同进入候选计划，并通过依赖合法性检查后再进行性能选择。

2.4 逻辑 Megakernel 的多 grid 物理映射

逻辑 task graph 与物理 grid 属于两个连续的执行计划层次。MPK 的 SM-level graph 和 Event Tensor 的依赖表示在逻辑层描述 task 及其 readiness；物理层再决定哪些相邻算子阶段共享 persistent grid 的执行状态。DeepGEMM MegaMoE 给出了算子图内部多 grid 映射的直接工程信号。open PR #360 的固定 head branch 实现了 SM90 cooperative MegaMoE，并把 dispatch、两层 GEMM、SwiGLU 与 combine 等阶段组织在一个 cooperative kernel 内；该 PR 的后续计划明确引用 green-context split 方案DeepSeek-AI (2026); mpdfdf (2026)。draft PR #357 的固定 head branch 将同一 MegaMoE 数据流分成 dispatch+L1+SwiGLU、L2+combine 和 combine-reduce 三个 focused kernels，前两个 kernel 使用 CUDA Green Contexts 占据互斥 SM partitions，并通过 HBM arrival masks 形成流水，整个路径由一个 CUDA Graph 重放RayWang96 (2026)。该 PR 在 B200 上报告 32 个配置与 fused 输

出逐位一致；8 ranks、8192 tokens、intermediate size 2048 的时间从 $2307.8\ \mu\text{s}$ 降至 $2046.7\ \mu\text{s}$ ，而多个中小 token 配置呈现 split 较慢的方向。其处理变量同时包含物理分区、focused-kernel 实现、Green Context 资源配额和 arrival-mask 流水，因此这些数据刻画复合多 grid 计划的 workload-dependent crossover；本课题的 Hazy slot-exact A/B 承担物理边界的单变量因果识别。

MegaMoE 的结果揭示了同一算子图中复合物理计划的 crossover：资源配额、producer-consumer overlap、focused kernel 与 grid 交接共同作用，其净收益随 token 数与中间维度改变。逻辑融合范围可以保持稳定，算子依赖的物理实现则在 single grid 与 multiple grids 之间选择。

2.5 CUDA Graph 与 Programmatic Dependent Launch

CUDA Graph 把 kernel、memcpy 与同步节点预先实例化为可重复执行的 DAG，从而摊销逐 kernel 的主机提交成本；默认 dependency edge 在上游节点完成后放行下游节点 [NVIDIA \(2026a\)](#)。这一机制降低了多 grid 计划的 host-side cost，并保留每个 grid 独立的资源配置和完成语义。DeepGEMM PR #357 使用单次 CUDA Graph replay 组织三个 focused kernels，展示了图重放与物理分区的直接组合。

Hopper 及后续架构提供 Programmatic Dependent Launch (PDL)。当 producer 的全部 blocks 分别显式触发 programmatic completion，或在退出时完成隐式触发后，consumer grid 获得提前调度机会；consumer 先执行不读取 producer 输出的独立前缀，并在 dependent read 前调用 `cudaGridDependencySynchronize()`。驱动对实际并发采用机会式调度，执行计划的正确性由同步语义独立保证 [NVIDIA \(2026d\)](#)。CUDA Graph 也可以用 programmatic edge data 表达同一依赖。

CUTLASS 将 PDL 集成到 GEMM launch API，并在 Example 63 中把 consumer GEMM 的独立权重预取放到 dependency synchronization 之前，使权重访问与 producer 尾部执行重叠 [NVIDIA \(2026b,c\)](#)。本课题前期公开代码 `pdl_exp` 在固定提交上完成了接口、构建路径以及 trigger/prefetch 参数化方式的校准；device-side PDL 指令与真实 overlap 由 CUTLASS Example 63 和 Hazy direct-PDL 实验确认 [Muuuchen \(2026\)](#)。因此，PDL 为物理边界增加了一个连续可调的交接空间：producer trigger 决定 consumer 最早投放点，consumer independent prefix 决定可隐

藏工作量，资源竞争决定实际 overlap 的净收益。

2.6 现状归纳与研究机会

现有研究已经形成三个相互衔接的优化层次。图级融合与 kernel orchestration 选择逻辑计算区域及其 primitive 组合；Megakernel compiler/runtime 选择 task 粒度、事件表示和设备端调度；CUDA Graph 与 PDL 优化多个物理 grid 的提交和交接。公开 artifact 进一步说明，同一个逻辑 Megakernel 可以映射为一个或多个物理 grid，而且两种映射的相对性能随依赖和 workload 改变。

由此得到本课题的研究变量。对逻辑 task graph 中的相邻算子阶段切口 $b = (V_A, V_B, E_b)$ ，定义依赖实现

$$\pi(b) \in \{\Pi_E, \Pi_G, \Pi_S, \Pi_P\}.$$

细粒度 event 保留 tile/head 级流水；global gate 在同一 persistent grid 内把每个 consumer 的 admission 对齐到其最小依赖 readiness，Hazy 中心边对应全量 readiness；grid completion 同时提供全局完成、内存可见和阶段生命周期切换；PDL 则把 consumer 的合法独立前缀前移到 producer 尾部。边界选择的净效应由消除的软件交接和阶段干扰、损失的细粒度流水、新增的 grid 边界成本以及 PDL 可隐藏工作量共同构成。

这一执行计划空间给出明确的研究机会：从依赖 fan-in/readiness、producer 与 consumer 时长、task admission 时序、worker 驻留、阶段资源包络和独立前缀中提取特征，为每个相邻算子阶段切口选择 $\pi(b)$ ；随后在选定物理边界上联合搜索 PDL trigger、prefetch volume 与 memory level。该问题把逻辑融合、物理分区和跨边界重叠连接为一条完整优化链，并以同图正负边、内部 gate 对照和 split+PDL 对照提供可验证的实验结构。

三 课题主要研究内容、预期目标

3.1 问题定义与总体目标

设已经生成的 Megakernel 任务图为 $G = (V, E)$ ，其中节点表示保持数值语义和 tile 实现不变的 GPU task。候选算子边界表示为 task graph 上的相邻阶段 cut

$$b = (V_A, V_B, E_b), \quad E_b = \{(u, v) \in E \mid u \in V_A, v \in V_B\},$$

其中 V_A 和 V_B 分别为 producer 与 consumer 算子阶段的 task 集合, E_b 为跨越该 cut 的数据依赖。对给定 workload x , 本课题研究边界 b 的**依赖实现层级**: 同一逻辑任务图可以由单个 persistent grid 内的软件事件承载, 也可以由 CUDA grid completion 承载; 选定物理边界后, 还可以利用 PDL 将后继 grid 的合法独立前缀与前驱尾部重叠。表 2 定义四种执行计划, 图 3 给出相应时间结构。

表 2 同一算子阶段切口的四种执行方案

记号	执行方案	依赖实现与可识别作用
Π_E	EARLY-INSIDE	V_A 与 V_B 位于同一 grid, 沿用原 Megakernel 调度器; consumer tasks 可以在其最小依赖 ready 前进入 worker queue。
Π_G	ADMISSION-GATED INSIDE	V_A 与 V_B 仍位于同一 grid; controller 在每个 consumer 的最小依赖满足后签发相应 task。中心边的最小条件为 32 heads 全量 ready。
Π_S	SERIAL-SPLIT	在 b 上形成两个同 stream 的物理 grids, 以 producer grid completion 完成全局 ready、数据可见和后继放行。
Π_P	SPLIT+PDL	保留 Π_S 的物理边界, 并通过 PDL 提前调度 consumer grid; dependency wait 前执行地址生成、descriptor 设置和独立数据预取。

四种计划构成三组相互衔接的因果对比。 Π_E 与 Π_G 的差值刻画 task admission 时序对 single-grid 关键路径的影响; Π_G 与 Π_S 的差值是物理边界的净残差, 其中包含 dispatch、suffix 初始化、consumer 执行时间、cache/locality、阶段资源状态和 grid 生命周期等分量; Π_S 与 Π_P 的差值刻画固定物理边界上 PDL 的净重叠收益。各组实验锁定输入、输出、精度、task 指令、tile/layout 和 task 到执行队列的映射; 物理边界引起的 device dispatch 与可见 GPU gap 作为显式成本进入测量, 使性能差值对应于完整的依赖实现决策。

记计划 Π_i 在边界 b 、workload x 上的延迟为 $T_i(b, x) = L(b, x; \Pi_i)$, 定义

$$\begin{aligned}
\Delta_{E \rightarrow G} &= T_G(b, x) - T_E(b, x), \\
\Delta_{G \rightarrow S} &= T_S(b, x) - T_G(b, x), \\
\Delta_{S \rightarrow P} &= T_P(b, x) - T_S(b, x).
\end{aligned} \tag{1}$$

式 (1) 将“是否拆分”细化为 task 投放、物理阶段切换和跨边界重叠三个可独立测量的决策。H800 前期结果已经在 Hazy 的同一任务图中给出一条 $T_S < T_E$ 的

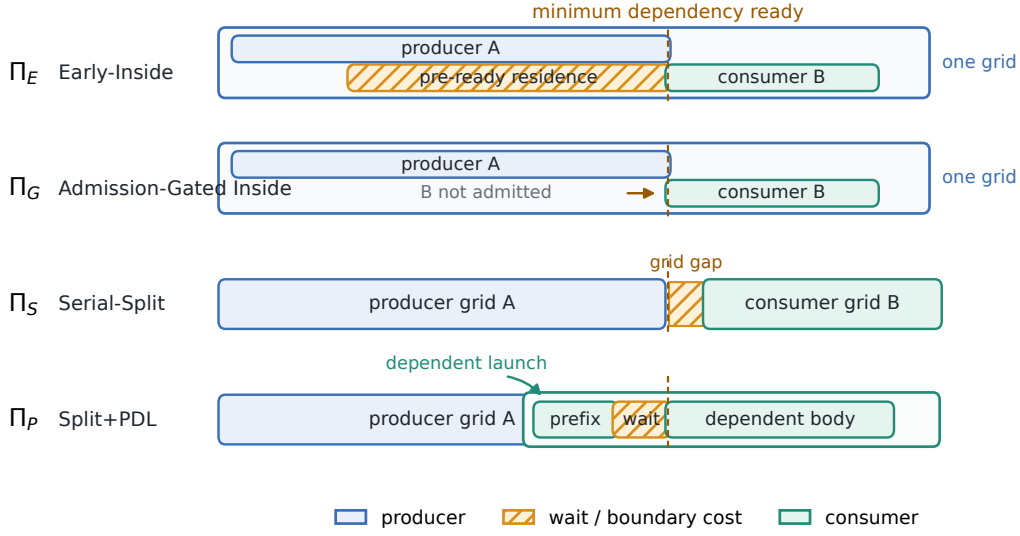


图 3 同一算子阶段切口的四种执行计划。 Π_E 允许 consumer 提前进入， Π_G 在最小依赖满足后投放 consumer， Π_S 以 grid completion 承载依赖， Π_P 在保留物理边界的同时提前执行合法独立前缀。三组相邻差值依次识别 task admission、物理阶段切换和 PDL 跨界重叠。

正边和三条反向边，并在正边上观察到 $T_P < T_S$ 。这些结果确立了边界选择的现实性；本课题进一步建立其因果机制、适用条件和端到端传递规律。

总体目标是建立**依赖感知的 Megakernel 物理分区与跨边界执行方法**：以依赖 ready 粒度、task 投放粒度、producer–consumer 时间结构和资源行为解释边界收益的正负转换；为选定物理边界生成满足数据依赖的 PDL 独立前缀；最终在真实 decoder layer 和 decode step 中形成可复现、可解释的执行计划选择依据。

3.2 研究问题

围绕上述目标，本课题回答以下四个研究问题。

1. **RQ1: 同一算子阶段切口应由哪一级执行机制承载？** 通过 Π_E 、 Π_G 和 Π_S 的受控比较，分离提前投放与物理边界净残差，并用阶段时间戳和主动干预预定位残差中的关键路径分量。
2. **RQ2: 哪些依赖与运行时特征决定物理分区的收益方向？** 研究全量或分块 ready、可流式 dependent work、合法 independent prefix、producer 窗口、跨边 locality、可见 launch gap 和阶段资源耦合对 crossover 的共同作用，以真实正负边和主动干预验证方向性预测。

3. **RQ3: PDL 在怎样的 trigger、预取量和共驻条件下产生净收益？** 对固定边界比较 Π_S 与 Π_P ，联合测量实际 overlap、producer slowdown、consumer tail 和完整临界路径，求得 PDL 的 break-even 区间。
4. **RQ4: 局部边界收益如何传递到真实 LLM 解码关键路径？** 将同一变换依次接入 isolated edge、decoder layer、完整 decode step 和短生成，量化收益保留率，并解释 critical-path position、context length、batch size 和调度重叠对收益幅度的影响。

3.3 可证伪研究假设

本课题以可主动改变的变量提出以下假设。每个假设同时给出预测与拒绝条件，实验结论由干预后的方向和置信区间决定。

H1: 依赖粒度与投放粒度错配。 当 V_B 的主要任务共享全量 ready 条件、ready 前的 dependent work 为零，且提前进入的 consumer 状态占用 issue、queue、cache 或同步资源，或由此延长 producer/handoff 关键路径时，预测 $T_G < T_E$ ；收益随 pre-ready producer 窗口及受影响的 worker-time 增大。若等待状态完全休眠且不改变 producer 与 handoff，则预测 $T_G \simeq T_E$ ，admission 项为零。对于按 tile/head 逐步 ready 且能够形成有效流水的边，预测 $T_E < T_G$ 。主动改变 readiness 粒度、admission 时机和 consumer 占用后，延迟及跨阶段时间戳的方向用于检验 H1。

H2: 物理边界的 break-even。 $\Delta_{G \rightarrow S}$ 是 dispatch gap、suffix 初始化、consumer 运行时间、cache/locality、阶段资源交互和 grid 生命周期共同形成的净残差。预测降低可见 GPU gap 将近似等量改善 T_S ，物化跨边中间数据将增加 locality 损失，资源状态与生命周期处理将改变 last-ready 前的 producer 路径或 ready 后的 handoff/drain。若分别改变这些因素后 $\Delta_{G \rightarrow S}$ 及其阶段时间分解始终保持原值，则 H2 被拒绝。

H3: PDL 的独立前缀与资源余量条件。 当 consumer 具有可在 dependency wait 前执行的独立前缀，producer 尾部提供足够重叠窗口，且被隐藏的前缀时间 H_{prefix} 超过 trigger、wait 和资源竞争开销之和时， $\Delta_{S \rightarrow P} < 0$ ；资源竞争项包含 PDL 引起的 producer slowdown。trigger 提前量与预取量增加时，隐

藏时间与三类开销形成可测的 crossover。若时间线显示 device-side overlap 为零，或独立前缀长度变化与净延迟变化持续解耦，则 H3 被拒绝。

H4：关键路径上的收益传递。 当目标边在 decoder layer 或 decode step 中持续暴露于关键路径时，上层延迟差值与 isolated edge 的差值同号，其幅度由该边在关键路径中的暴露比例决定。若 Nsight Systems 确认目标边持续暴露于关键路径，而独立重复的 layer/decode 延迟置信区间稳定为零或反向，则 H4 被拒绝。

H1 与 H2 的前期消融已经提供两项约束：将 first-TMA 移到 ready 后以及将 polling backoff 从 1 ns 增至 1024 ns 时，Hazy 的 split 差距仍稳定在 44.832–47.776 μ s。因此，后续因果实验聚焦 controller-level task admission、last-producer-ready 之前的 producer 关键路径，以及 ready 后的 VM/controller handoff 与 grid drain；同一时钟域的设备端时间戳定位差值所在阶段，admission、resource 和 lifecycle 主动干预进一步识别因果机制。

3.4 主要研究内容

1. **建立四方案受控基线并分解关键路径。**在 Hazy 真实正边及同图负边上实现 Π_E 、 Π_G 、 Π_S 和 Π_P 。使用同步 %globaltimer 记录 last producer store/atomic、programmatic trigger、consumer admission/grid start、producer exit、dependency wait return、first dependent read、consumer completion 与 consumer grid exit，以式 (1) 定位收益来源；A/A、独立参考实现和随机交错测量共同控制数值正确性与时钟漂移。
2. **构建依赖—投放—边界 crossover 模型。**从源码与执行 trace 中提取 ready 粒度、dependent/independent work 比例、producer 窗口、跨边复用、worker admission 和真实 GPU gap，并通过改变 context、task 粒度、ready 结构和 admission 时机进行主动干预。真实 Megakernel 正负边承担外部有效性，受控子图承担单变量机制检验；二者共同形成可解释的边界适用域。
3. **实现安全的 PDL 跨边界变换。**对 producer–consumer 边建立读写集合、alias、同步支配和 progress-safety 分析，从 consumer 中提取可提前执行的独立前

缀；联合搜索 programmatic completion trigger、预取数据量和目标内存层级，并以 T_S 为基线量化 overlap 收益与 producer 干扰的平衡点。

4. **验证真实解码中的传递性并形成选择方法。**将候选边接入 decoder layer、decode step 和短生成，在 context length、batch size 与模型 shape 上报告 P50/P90/P99、TPOT 和关键路径分解。基于多个真实正负边冻结特征卡与方向性规则，再在 held-out 边上检验边界选择和 PDL 参数选择，形成可嵌入 Megakernel 编译器与运行时的原型。

3.5 预期目标、创新点与成果

3.5.1 预期目标

1. 完成 Hazy 中至少一条正边和一条负边的机制闭环，量化 pre-ready admission、producer-side interaction、ready 后 handoff、边界 gap 与内部流水在关键路径中的作用，并给出 T_G 与 T_S 的直接判定结果。
2. 在第二类真实 Megakernel 计算图和受控子图中复验 H1-H2，获得覆盖 coarse/fine readiness、短/长 producer 和不同 locality 的正负样本，形成边界 crossover 的适用域。
3. 建立 PDL 合法性检查和 trigger-prefetch 搜索，得到 T_P 相对 T_S 的 break-even 曲线，并用 timeline 解释收益、饱和与反转区域。
4. 完成 isolated edge 到 decoder layer、decode step 与短生成的传递实验，形成带有置信区间、正确性记录和 profiler 证据的可复现实验包。

3.5.2 预期创新点

1. 提出“依赖 ready 粒度—task 投放粒度—物理执行粒度”的 Megakernel 边界 crossover 模型，用同一任务图上的正负边和主动干预解释软件依赖与 grid completion 的适用条件。
2. 建立 Π_E 、 Π_G 、 Π_S 、 Π_P 四计划统一比较空间，将内部投放控制、物理阶段切换和 PDL 边界回收组织为连续的执行计划选择过程。

3. 提出面向已选物理边界的依赖安全 PDL 变换，以独立前缀、同步支配与 producer interference 共同决定 trigger 和预取配置，并将局部变换的收益传递纳入端到端选择依据。

3.5.3 预期成果

预期形成一套 Megakernel 边界基准与设备端时间线工具、一个依赖感知的物理分区与 PDL 优化原型、一组覆盖真实正负边及层级传递的可复现实验数据，以及完成硕士学位论文和高水平学术论文投稿所需的理论、系统与实验结果。

四 拟采用的研究方法、技术路线、实验方案及其可行性分析

4.1 问题建模与分析对象

本课题以已经生成的 Megakernel task graph 为输入，并沿用第 3.1 节对算子阶段切口 $b = (V_A, V_B, E_b)$ 的定义。 V_A 与 V_B 是切口两侧的 task 集合， E_b 是跨越切口的全部依赖。实验保持逻辑 task graph、算子实现、数值精度和 task 到执行队列的映射不变，仅改变切口 b 的依赖实现层级。表 2 定义的 Π_E 、 Π_G 、 Π_S 和 Π_P 构成统一对照；相应实测延迟记为 T_E 、 T_G 、 T_S 和 T_P 。分析直接采用式 (1) 的三个差值。

$T_G - T_E$ 识别 single-grid 内 task admission 时序的影响； $T_S - T_G$ 是用物理 grid completion 替代 readiness-matched internal gate 后的净残差； $T_P - T_S$ 识别固定物理边界上 PDL 的净影响。Hazy 前期实验已经测得中心边界满足 $T_S - T_E < 0$ 。前三路对照将该合量精确分解为 admission 与物理边界两部分；第四路在既有 split 上继续测量 PDL 增量。

原始 single-grid 与物理分区之间满足精确恒等式

$$T_S - T_E = \underbrace{(T_G - T_E)}_{\Delta_{\text{admission}}} + \underbrace{(T_S - T_G)}_{\Delta_{\text{physical}}}. \quad (2)$$

$\Delta_{\text{admission}}$ 由同一 grid 内的 gate 干预直接测得。 Δ_{physical} 保留设备 dispatch、GPU gap、suffix 初始化、consumer 执行、cache/locality、资源状态和 grid 生命周期的共同净效应。后续实验按照预先固定的干预顺序逐项改变 gap、materialization、资源包络和 grid 生命周期，并同时报告交互项；由此避免把同一 consumer/cache

变化重复记入多个加和项。物理分区的充要判据为 $\Delta_{\text{admission}} + \Delta_{\text{physical}} < 0$; PDL 进一步改善完整计划的判据为 $T_P - T_S < 0$ 。

4.2 四路因果对照与关键路径定位

4.2.1 Controller-level admission gate

ADMISSION-GATED INSIDE 是识别 admission 作用的核心干预。一般情况下, controller 在每个 consumer 的最小输入依赖满足后发布该 task; 具有 tile/head 级依赖的边按相应局部 ready 条件释放。在 Hazy 中心边上, 每个 O-projection task 都读取完整的 32-head Attention 输出, 因此 gate 使用 32 heads 全量 ready 条件。controller 在创建 consumer page、semaphore 并向 instruction ring 发布 O-projection task 以前执行该检查, 使 queues 8–131 在 producer-ready 前继续原有 prefix/noop 路径, 而不创建 consumer 执行状态。consumer task、queue 目标、instruction bytes、TMA pipeline、数学运算以及整个 grid 的 launch 配置均与 EARLY-INSIDE 保持一致。

这一对照把“同一 grid”与“提前投放”两个因素分离。若 $T_G < T_E$, 差值由 readiness 前后的 admission 时序改变产生。若 $T_S < T_G$, 该结果首先识别出物理边界相对 internal gate 的净收益; 设备 dispatch、suffix 初始化、consumer 执行、cache/locality、资源状态和 grid 生命周期的贡献再由全局时间戳及逐项干预分解。两个差值可以同时出现, 且在同一匹配构建中满足 $(T_G - T_E) + (T_S - T_G) = T_S - T_E$ 。

4.2.2 跨阶段全局时间戳

每个方案使用 %globaltimer 在统一 GPU 时间域记录共同的 task 事件: producer 首个 task 开始、最后一个输出 store、每个 consumer 的最小依赖 ready、internal admission、首次依赖读取、最后一个 producer task 完成和最后一个 consumer task 完成。对关键路径 consumer k 定义

$$\begin{aligned}
 R &= t_{\text{ready},k} - t_{\text{Astart}}, \\
 H &= t_{\text{dep},k} - t_{\text{ready},k}, \\
 C_k &= t_{\text{Bdone},k} - t_{\text{dep},k}, \\
 D_{A,\text{stage}} &= t_{\text{Adone}} - t_{\text{ready},\text{last}}.
 \end{aligned} \tag{3}$$

其中 $t_{\text{ready},k}$ 是 consumer k 的 readiness-matched 时刻；Hazy 中心边的所有 $t_{\text{ready},k}$ 均等于 32 heads 全量 ready。 R 表示到达逻辑 ready 的 producer 路径， H 表示 ready 后到首次依赖读取的交接， C_k 表示 consumer k 的 dependent computation， $D_{A,\text{stage}}$ 表示最后一次逻辑 ready 后仍留在 producer 阶段的 task 尾部。

serial split 与 PDL 额外记录 programmatic trigger、consumer grid start、dependency wait 进入与返回、producer grid exit 和 consumer grid exit，并定义

$$\begin{aligned} P_{\text{ind}} &= t_{\text{wait enter}} - t_{B\text{start}}, \\ W_{\text{dep}} &= t_{\text{wait return}} - t_{\text{wait enter}}, \\ O_{\text{grid}} &= \max(0, t_{A\text{grid exit}} - t_{B\text{grid start}}), \\ D_{B,\text{post}} &= t_{B\text{grid exit}} - t_{A\text{grid exit}}. \end{aligned} \tag{4}$$

P_{ind} 、 W_{dep} 、 O_{grid} 和 $D_{B,\text{post}}$ 分别刻画 PDL 的独立前缀、显式等待、grid 驻留重叠和 producer 退出后的 consumer 尾部。single-grid 计划记录 t_{Adone} 与 whole-grid exit；其语义与 producer grid exit 分开报告。PDL 下多个区间在时间轴上重叠，因此完整 envelope 由每个样本的起止端点直接计算，并同时报告 producer duration 与关键路径变化。instrumented binary 用于事件排序和阶段分解，最终时延由无 instrumentation 的匹配构建测量。

4.2.3 SM 调度与指令级解释

Nsight Systems 用于测量完整 envelope、两个 grids 的实际起止时刻、可见 gap 以及 PDL overlap；Nsight Compute 用于分析单个 kernel 的 SM 和指令行为。重点指标包括 active/eligible warps、issue slot 使用、barrier 与 memory dependency stall、L1/L2 sectors、DRAM bytes、TMA 请求、寄存器、shared memory 和 achieved occupancy。SASS 审计用于确认 polling loop、griddepcontrol、TMA 位点与 consumer 首次依赖 load 的静态顺序。

机制结论由“源码依赖事实、动态时间顺序、主动干预、关键路径变化”四层证据共同给出。以 polling 为例，backoff 处理改变轮询 load 数量，admission gate 改变 consumer 是否被发布，二者分别检验 memory transaction 数量和 task admission 位置。NCU replay 得到的 kernel duration 和 counter 用于解释硬件行为；端到端差值统一使用无 profiler 的成对计时和包含 grid gap 的 Nsys envelope。

4.3 边界特征与依赖实现层级选择

本课题为每个相邻算子阶段切口构建 feature card, 特征分为四组。第一组来自编译期静态分析, 包括 producer/consumer 读写集合、ready counter 拓扑、consumer 在 full-ready 前可执行的依赖工作比例、task fan-in/fan-out、跨边中间张量大小、数据驻留层级以及两个阶段的寄存器和 shared-memory envelope。第二组来自原始计划的低开销 trace, 包括 producer window、早到 worker 数、ready 到首次依赖读取的 handoff 和阶段尾部。第三组来自主动干预, 包括 admission delay、producer length、consumer fanout、局部 readiness 粒度和 locality materialization; 物理边界成本由 C++/CUDA Graph 提交路径校准以及候选 split 的实测 GPU gap 给出。第四组 profiler 特征用于解释最终选择, 并与选择器训练和评测数据分离。

对切口 b 定义 full-ready 前可利用工作比例

$$\phi_b = \frac{\text{consumer 在 producer full-ready 前可完成的依赖计算量}}{\text{consumer 总依赖计算量}}. \quad (5)$$

ϕ_b 接近 1 表示局部 tile/head readiness 能形成充分流水; $\phi_b = 0$ 表示 dependent computation 具有全量、全或无的就绪条件。 ϕ_b 与 producer window P_b 、提前投放 fanout Q_b 、跨边局部性 L_b 、资源 envelope 差异 U_b 和真实边界成本 C_b 共同决定执行方案。选择器首先以可解释的分段模型预测式 (1) 的符号和量级, 再由少量硬件测量校准靠近 crossover 的候选。

样本按算子边和计算图划分训练、校准与 held-out 集。边界收益、NCU 诊断量和人工处理后的结果变量只进入模型评估, 静态特征与低开销基线 trace 构成部署时输入。该划分使边界判断对应实际编译与运行时可获得的信息。

4.4 PDL 跨边界变换与收益模型

物理分区确定后, consumer 阶段 V_B 被划分为独立前缀 B^{ind} 和依赖主体 B^{dep} 。独立前缀的合法性条件为

$$\begin{aligned} R(B^{\text{ind}}) \cap W(V_A) &= \emptyset, \\ W(B^{\text{ind}}) \cap (R(V_A) \cup W(V_A)) &= \emptyset. \end{aligned} \quad (6)$$

其中读写集合包含指针 alias 的传递闭包; 提前写入的数据位于 consumer-private 存储或与 producer 读写集合互斥的区域。consumer 的 dependency wait 支配 B^{dep} 中每一次对 V_A 输出的读取; producer 的全部 CTAs 分别显式 trigger, 或在退出

时完成隐式 programmatic completion；内存可见性由 consumer wait 建立。独立前缀优先容纳地址计算、descriptor 设置、指令解析和 immutable weight staging。

PDL 配置表示为 $\langle q, p, h \rangle$ ： q 为 producer trigger 阶段， p 为 consumer 预取量， h 为寄存器、shared memory 或 cache 等目标层级。其净差值写为

$$\Delta_{\text{PDL}} = I_{\text{trigger}} + I_{\text{wait}} + I_{\text{competition}} - H_{\text{prefix}}, \quad (7)$$

H_{prefix} 表示被隐藏的 consumer 独立前缀与普通边界 gap， $I_{\text{competition}}$ 表示提前 consumer 与 producer 共驻引起的资源干扰。对 q, p, h 的扫描同时报告 producer duration、consumer post-ready tail、实际 overlap 和完整 envelope，从而选取满足 $\Delta_{\text{PDL}} < 0$ 的配置。

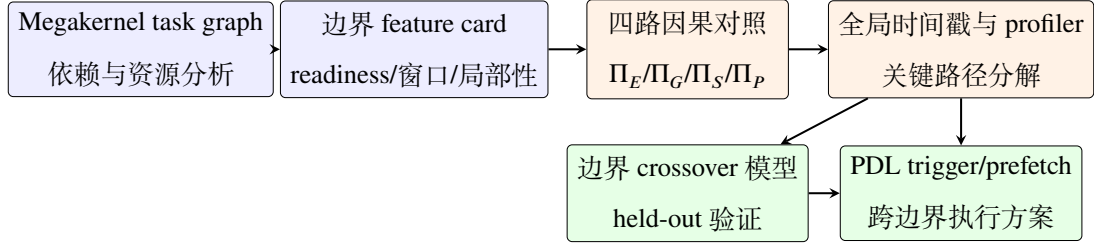


图 4 本课题的技术路线：以算子阶段切口为分析对象，通过四路主动干预定位 crossover 机制，并在选定物理边界上优化 PDL 独立前缀。

4.5 实验方案

4.5.1 实验层次与真实计算图

实验按照“算子边、decoder layer、decode step、短生成”四个层次展开。Hazy Research Megakernels 的 layer-0 isolated operator region 作为完整四路因果实验的主载体，中心边界覆盖长 KV context 下的 PartialAttention 与 O-projection；同图相邻负边用于验证选择方向。第二类真实计算图从 MegaQwen、Mirage MPK/Event Tensor 或 DeepGEMM MegaMoE 中选择可在 H800 上保持原始算法和调度语义的候选。第二图分别实现原始 inside、该架构能够表达的最强 readiness-matched internal gate、serial split 与 split+PDL，并记录其与 $\Pi_E - \Pi_P$ 的语义对应关系。每个真实图先冻结 commit、checkpoint、shape、task 实现和基线 schedule，再生成匹配的边界方案。

Workload 扫描 batch、context/KV length、producer task 数和 consumer fanout。Hazy 主实验覆盖 batch 1 与 4K–131K context，并在边界 crossover 附近加密采样。

若切口 b 在一次 decode 测量中动态出现 N_b 次，则 layer/decode 实验报告局部差值传递到 TPOT 的保留率

$$\rho = \frac{T_{\text{decode,base}} - T_{\text{decode,opt}}}{\sum_{j=1}^{N_b} (T_{\text{edge,base}}^{(j)} - T_{\text{edge,opt}}^{(j)})}, \quad (8)$$

重复边形状相同时，分母可写为 $N_b \Delta T_{\text{edge}}$ 。实验同时测量连续 token 生成中的 P50、P90 和 P99；P99 使用至少 10^4 个稳态样本并报告样本数。

4.5.2 受控机制子图

受控子图用于独立操纵真实系统中彼此耦合的因素。第一组保持 single-grid, 构造

global/chunk readiness $\times$ short/long producer $\times$ early/gated admission

的 $2 \times 2 \times 2$ 实验，检验全量就绪、producer window 与 task admission 的交互。第二组在已经选定的物理边界上构造

serial/PDL $\times$ trigger phase $\times$ prefetch amount,

测量 PDL 的 break-even。两个实验分别回答内部依赖实现和外部边界交接，结果再由真实计算图验证。

4.5.3 严格 A/B 与统计方法

边界 A/B 固定 producer/consumer task code、输入、输出、dtype、SM queue 映射、真实 task/noop slot、instruction bytes、dependent computation 顺序和数值运算顺序。inside instruction tensor 等于 prefix 与 suffix tensor 的逐字节拼接；serial 与 PDL 调用同一 producer/consumer device symbols，合法独立前缀的跨 grid 时序和 launch attribute 构成 PDL 的处理变量。每个构建保存源码、二进制、SASS、checkpoint 和 trace 的 hash。

正确性依次检查 barrier/counter、关键中间张量、最终 hidden state 和独立 PyTorch reference。运算顺序固定的路径使用逐位比较；原算法含原子归约顺序变化的路径使用预先定义的 BF16/FP32 误差界。压力测试随机交错各执行方案，以覆盖 PDL trigger、wait 和多轮复用。

性能实验以“物理 GPU-日期-独立进程”作为分层统计单位。每个进程完成 warmup 后执行随机 AB/BA 或多方案随机排列，进程内重复首先汇总为 paired median delta；推断阶段按 GPU、日期和进程分层重采样，给出 95% 置信区间，并逐一报告独立单元的方向、绝对微秒差、相对变化和 paired win rate。进程内 bootstrap 仅刻画该进程的重复测量噪声。A/A 随机标签、cache eviction 和 GPU timeline gap-inclusive 对照用于量化计时噪声、缓存状态与 host submission 的影响。

4.5.4 对照与消融

核心对照包括原始 EARLY-INSIDE、ADMISSION-GATED INSIDE、SERIAL-SPLIT、SPLIT+PDL，以及最终执行计划的 CUDA Graph 重放。消融依次改变 readiness 粒度、producer window、consumer fanout、first-TMA 时序、polling backoff、cross-edge materialization、PDL trigger、prefetch amount 和 storage level。每项源码干预同时应用于 inside 与 serial control，并报告差分中的差分

$$(T'_S - T'_E) - (T_S - T_E),$$

从而分离代码处理的固定开销与该因素对 split gain 的交互效应。

4.6 可行性分析与风险控制

本课题的中心 A/B 已在两张 H800 上打通 task 生成、CUDA 构建、真实 checkpoint、成对计时、Nsight Systems、Nsight Compute 和 SASS 审计链路。Hazy 边界具备一个稳定正样本和三个方向相反的同图样本；一张物理 H800 上的修改后匹配 device binary 已在同一正边形成 serial/PDL 对照。上述基础支持对式 (1) 进行后续分解。

主要实验风险对应三项控制。第一，instrumentation 可能扰动 SM 调度；动态时序和最终时延分别由 instrumented 与无 instrumentation 构建承担，并比较资源/SASS 变化。第二，host submission 可能放大短 producer 的分区成本；C++ launch path、CUDA Graph 与 Nsys GPU gap 将区分软件提交和设备边界。第三，边界样本数量决定选择模型的复杂度；早期采用可解释分段模型，特征和阈值在 held-out 图测试前冻结，跨图数据增长后再扩展搜索空间。上述安排使每个阶段都能输出独立可检验的机制结论和系统结果。

五 已有科研基础与所需的科研条件

5.1 真实 Megakernel 的实验基础

本课题以 Hazy Megakernels 作为可复现研究载体，版本和固定提交见公开仓库[Hazy Research \(2025b\)](#)。目标代码采用 Hopper 低时延 Llama 路径，实验复用上游 Attention、MatVec 与通用 VM task code。模型权重采用公开、同形的 Llama checkpoint，其版本由公开实现固定[inference-optimization \(2025\)](#)。连续自然文本 token 序列由冻结的输入生成脚本产生。主实验中的一个 grid 包含 132 CTAs，每个 CTA 640 threads；worker 在运行时读取 SMID 并选择对应 instruction queue。

中心 A/B 将 PartialAttention $\rightarrow$ O_ProjResidual 实现为一次或两次 mk_llama launch。inside instruction tensor 等于 prefix 与 suffix tensor 在 slot 维的逐字节拼接；两种方案均包含 136 个真实 tasks、128 个 noops 和相同的 33,792 instruction bytes。attention output、hidden output 和 barrier 逐位一致，并通过独立 PyTorch reference。该控制使性能处理变量对应相同 isolated task stream 中间是否放置物理 grid boundary。

表 3 汇总先导实验的控制条件与可识别结论。131K context 下，原始 Hazy binary 的三个独立进程均得到 45.42–46.64 μ s 的成对净节省；32K、65K、98K、120K 和 131K 扫描显示绝对差值随长 attention producer 从约 7–8 μ s 增长到约 45–47 μ s。同一图的四条相邻边呈相反方向，构成边界选择的正负样本。

图 5(a) 给出冻结 task stream 下长 context 的绝对收益范围。context 从 65K 增至 131K 时，物理分区节省从约 20 μ s 增至约 46 μ s；120K Nsys 中可见 grid gap 为 1.056 μ s。图 5(b) 将同一 Hazy 计算图的四条相邻边统一到 32K workload：中心边的三个独立进程均支持物理分区，其余三条边均支持 single-grid 执行。绝对收益随 producer window 增长，而边界方向随依赖结构变化；两项观察共同确定后续 single-grid admission 与跨阶段交接实验的量级和对照组。

5.2 SM、SASS 与关键路径分析基础

源码审计已经确定中心边的依赖结构：8 个 PartialAttention tasks 产生 32 个 query heads，128 个 O-projection tasks 各自读取完整的 attn_out[2048]。SASS 中已经定位 global ready counter 的 system-scope load、compare、nanosleep 回环、CTA barrier、weight TMA 和首次 activation load。逐 SM 时间戳进一步确认 queues 8–131

表 3 H800 先导实验的控制条件与直接结论

实验资产	已实施控制	可识别结论
Slot-exact 中心边	固定 task code、queue 映射、输入输出、task/noop 位置和 33,792 instruction bytes	原始 single-grid 与 serial split 的净差值来自物理映射变化
数值与同步正确性	Attention、hidden、barrier 逐位比较；独立 PyTorch reference；随机交错复用	A/B 保持算子数值语义与跨阶段依赖语义
独立重复	131K 三个独立进程各 100 对；32K 中心边在第二张 H800 上复验；计时与 profiler 分离	正边方向跨进程、跨卡保持；进程内噪声由 A/A 量化
动态 task 顺序	124 个早到候选 queues 的 372/372 次观测均在 full-ready 前进入依赖等待	全量 readiness 与 SM-local 提前推进在真实执行中同时发生
first-TMA $\times$ backoff	两张 H800、三个独立进程、四个 cells，共 12 个 paired medians	ready 前 weight TMA 与 polling load 数量均非维持 44.832–47.776 μ s 差距的单一主因
同边 Split+PDL	一张物理 H800；修改后的匹配 binary；serial/PDL 仅改变 programmatic launch 路径	PDL 在该边进一步节省 2.4–3.8 μ s，并形成真实 grid overlap

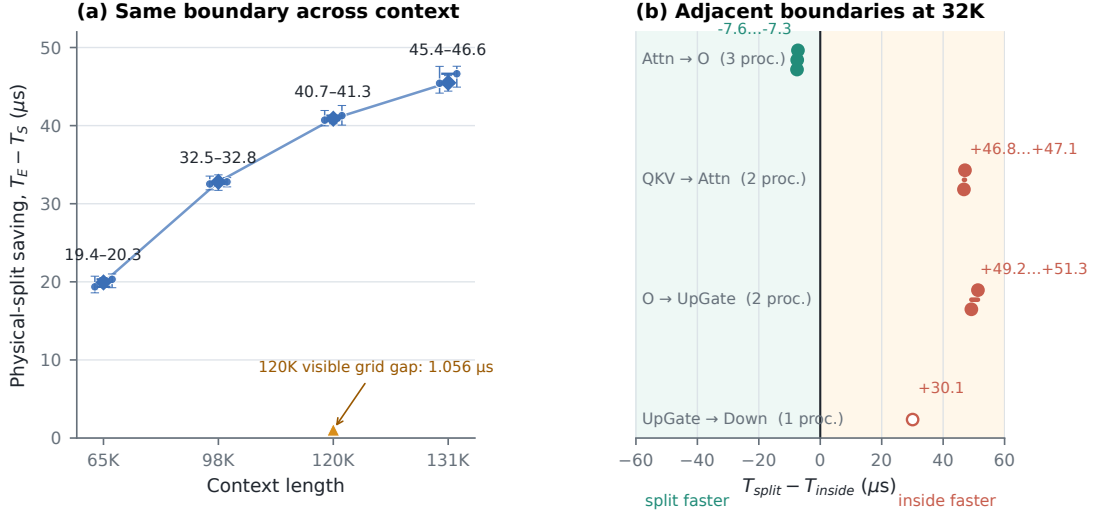


图 5 H800 上的物理边界实测结果。(a) 蓝色小点与细误差线表示各独立进程的 **paired median** 和进程内 **paired-bootstrap 95%** 区间，菱形与粗短线表示进程间中位数和 **min-max** 范围；正值表示 **Serial-Split** 更快。橙色三角为 120K Nsight Systems 单独测得的可见 **inter-grid gap**。(b) 同一 32K workload 上四条相邻边的进程级 **paired median**；绿色区域对应 **split** 更快，红色区域对应 **inside** 更快，空心点表示该边当前包含一个独立进程。

的实际提前进入与 first-TMA 顺序。

Nsight Compute 在 120K/131K 观察到 **inside** 的 L1 global-load sectors 随 context 增长至 **outside** 的 $97.6 \times / 115.75 \times$ ，而两种方案的总 DRAM bytes 基本相同。将 polling backoff 从 1 ns 增至 1024 ns 后，额外 L1 global-load sectors 下降 88.1%，split active-time gain 基本保持；将 first-TMA 移至 full-ready 后，131K 的完整 split gain 同样保持。两项干预表明这些指标是原始路径的机器行为特征，其单独变化不足以关闭性能差距；controller admission、producer pre-ready 路径与 ready 后交接由下一阶段时间戳继续分解。

Nsight Systems 已建立 gap-inclusive envelope 测量。120K 中，inside envelope 为 $2922.359 \mu s$ ；outside 的 prefix、gap 和 suffix 的边际中位数分别为 $2872.711 \mu s$ 、 $1.056 \mu s$ 和 $8.048 \mu s$ ，由逐样本起止端点直接计算的完整 outside envelope 中位数为 $2881.783 \mu s$ 。该时间线直接量化了可见物理边界成本与完整计划差值。

5.3 PDL 与跨代码库实现基础

本课题已经在一张物理 H800 上将 PDL 直接叠加到 Hazy 中心边。ordinary 与 PDL 两种分区方案使用同一个修改后匹配 binary，并调用相同的 pro-

ducer/consumer device function；处理变量是 suffix launch 的 programmatic stream serialization attribute。producer 的每个 CTA 发出 completion trigger, consumer 在首次读取 attn_out 前执行 grid dependency wait, wait 前只读取 immutable O weights 并完成私有 shared-page staging。32K 和 131K 各三个独立进程的 paired-median bootstrap 区间均支持 PDL 的净正收益。Nsys 记录到 20/20 个 PDL pairs 发生 overlap, 并将 producer 完成后的 suffix tail 从 $9.088\ \mu\text{s}$ 缩短到 $5.008\ \mu\text{s}$ 。

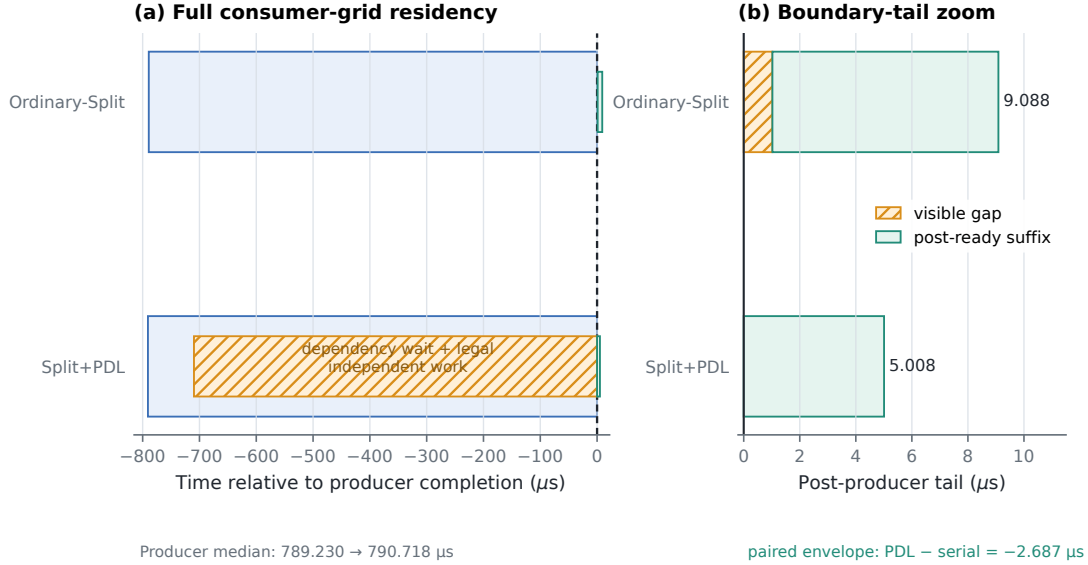


图 6 Hazy 中心边上 Serial-Split 与 Split+PDL 的 Nsight Systems 时间结构 (32K, matched binary, 20 对 AB/BA)。左图将每种方案的 producer completion 对齐为零, 斜线区表示 dependent grid 在 producer 完成前的驻留, 其中包含 dependency wait 与合法独立工作; 右图放大 producer completion 后的关键路径尾部。PDL 将 suffix tail 中位数从 $9.088\ \mu\text{s}$ 降至 $5.008\ \mu\text{s}$, 完整 GPU envelope 的 paired-median 变化为 $-2.687\ \mu\text{s}$ 。

代码调研与运行路径覆盖 MegaQwen、DeepGEMM MegaMoE、Mirage MPK、NVIDIA CUTLASS dependent launch 示例以及已有 PDL 实验仓库。MegaQwen 与 DeepGEMM MegaMoE 采用公开仓库的固定版本 [Infatoshi \(2026a\)](#); [DeepSeek-AI \(2026\)](#); Mirage MPK 采用论文公开版本 [Cheng 等 \(2026\)](#); PDL 接口依据 CUTLASS 示例和现有实验实现 [NVIDIA \(2026b\)](#); [Muuuchen \(2026\)](#)。其中, pdl_exp 用于接口、构建和参数路径校准; CUTLASS Example 63 与 Hazy direct-PDL binary 提供 device-side PDL 指令和真实 overlap 证据。MegaQwen 的 Qwen3-0.6B cooperative decode 已在 H800 完成可行性运行; MegaQwen MLP 真实 checkpoint trace 上已建立同 producer/consumer binary 的 serial-PDL 对照; DeepGEMM SM90 cooperative

MegaMoE 已完成本机构建与官方 smoke correctness。上述代码路径为第二真实计算图 and 不同调度结构的交叉验证提供候选载体。

5.4 软硬件与实验条件

当前实验节点配备 4 张 NVIDIA H800 80GB GPUs (132 SM, compute capability 9.0), 驱动版本 535.161.08; CUDA 13.2、Nsight Compute、Nsight Systems、cuobjdump、PyTorch 和完整 CUDA/C++ 构建环境已经可用。现有实验使用其中两张物理 H800 独立复验中心结果, 其余设备可用于跨卡方差、并行构建和 profiler 实验。

实验资产已经包含固定上游 commits、公开 checkpoint、自然 token traces、源码与二进制 hash、SASS、逐样本 CSV/JSON、Nsys reports、NCU reports、A/A controls、正确性脚本和汇总脚本。后续工作直接在这些冻结资产上增加 admission-gated variant、layer/decode integration 和 held-out graph, 能够保持先导实验与后续机制验证的可比性。

六 研究工作计划与进度安排

研究工作按照“机制闭环、边界规律、跨边界优化、真实路径验证、执行计划选择”的顺序推进。阶段安排与可验收成果如下。

表 4 研究进度安排与阶段成果

时间	主要工作	阶段成果
2026.07–2026.08	在 Hazy controller 发布 consumer instruction 前实现 full-ready admission gate; 加入同步 %globaltimer 里程碑; 完成 Π_E 、 Π_G 、 Π_S 三路 matched A/B	producer-ready、post-ready handoff 和 grid drain 的关键路径分解
2026.09–2026.10	扫描 readiness 粒度、producer window、consumer fanout、cross-edge locality; 完成 single-grid $2 \times 2 \times 2$ 受控子图	边界 crossover 的因果曲线与第一版 feature card
2026.11–2026.12	将中心边界接入 decoder layer 与 decode step; 建立 C++/CUDA Graph launch 路径; 测量连续 token 和短生成	局部收益保留率、TPOT/P90/P99 及 host-gap 分解
2027.01–2027.02	联合搜索 PDL trigger、独立前缀、预取量和存储层级; 测量资源竞争与前向进展; 扩展到第二真实 Megakernel 图	PDL break-even 模型、跨图正负样本和完整正确性结果
2027.03–2027.04	冻结边界静态/trace 特征与分段成本模型; 在 held-out 边和 held-out 计算图上评测; 完成主要消融	依赖实现层级选择方法、预测准确率和硬件测量开销
2027.05–2027.06	整理系统原型、实验 artifact、图表与论文; 完成开源复现说明和学位论文撰写	可复现实验包、论文投稿稿件和学位论文

参考文献

- Chen T, Moreau T, Jiang Z, et al. TVM: An automated end-to-end optimizing compiler for deep learning [C/OL]//13th USENIX Symposium on Operating Systems Design and Implementation. 2018: 578-594. <https://www.usenix.org/conference/osdi18/presentation/chen>.
- Cheng X, Zhang Z, Zhou Y, et al. MPK: A compiler and runtime for mega-kernelizing tensor programs [C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26). Seattle, WA: USENIX Association, 2026: 1909-1926. <https://www.usenix.org/conference/osdi26/presentation/cheng>.
- Dao T. Flashattention-2: Faster attention with better parallelism and work partitioning [C]//The Twelfth International Conference on Learning Representations. 2024.
- Dao T, Fu D Y, Ermon S, et al. Flashattention: Fast and memory-efficient exact attention with IO-awareness [C]//Koyejo S, Mohamed S, Agarwal A, et al. Advances in Neural Information Processing Systems: volume 35. Curran Associates, Inc., 2022: 16344-16359.
- DeepSeek-AI. DeepGEMM: Clean and efficient FP8 GEMM kernels [EB/OL]. 2026[2026-07-20]. <https://github.com/deepseek-ai/DeepGEMM/tree/559d79fb6994a58b8a15b4b93bf13ccc16edf247>.
- Dong W, et al. Ada-mk: Adaptive megakernel optimization via automated DAG-based search for LLM inference [EB/OL]. 2026. <https://arxiv.org/abs/2605.11581>.
- Hazy Research. Look ma, no bubbles! designing a low-latency megakernel for LLM inference [EB/OL]. 2025[2026-06-30]. <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>.
- Hazy Research. Megakernels: Low-latency LLM inference with device-side task graphs [EB/OL]. 2025[2026-07-20]. <https://github.com/HazyResearch/Megakernels/tree/7309cec801537b61fea3b50d7dfe454a6cde578e>.
- Hu M, Venkatram A, Biswas S, et al. Optimal kernel orchestration for tensor programs with Korch [C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. ACM, 2024: 755-769. <https://doi.org/10.1145/3620666.3651383>.
- Infatosh. MegaQwen: Custom CUDA megakernel for Qwen3-0.6B inference [EB/OL]. 2026[2026-07-20]. <https://github.com/Infatosh/MegaQwen/tree/0bc1fdedb82744481373000c5c7d3c58795c7a3a>.
- Infatosh. MegaQwen experiment results [EB/OL]. 2026[2026-07-20]. <https://github.com/Infatosh/MegaQwen>.
- inference-optimization. Llama-3.2-0.5B-Instruct checkpoint and model configuration [EB/OL]. 2025[2026-07-20]. <https://huggingface.co/inference-optimization/Llama-3.2-0.5B-Instruct>.

- Jaber J, Jaber O. AutoMegaKernel: A statically-checked agent harness for self-retargeting megakernel synthesis [EB/OL]. 2026. <https://arxiv.org/abs/2606.09682>. DOI: 10.48550/arXiv.2606.09682.
- Jin H, Hou B, Wang G, et al. Event tensor: A unified abstraction for compiling dynamic megakernel [C/OL]//Proceedings of Machine Learning and Systems: volume 8. Bellevue, WA, USA, 2026 [2026-07-20]. <https://openreview.net/forum?id=PJqFhAbUHa>.
- Luo X, Liu Z, Zhou Y, et al. Clusterfusion: Expanding operator fusion scope for LLM inference via cluster-level collective primitive [C]//Belgrave D, Zhang C, Lin H, et al. Advances in Neural Information Processing Systems: volume 38. Curran Associates, Inc., 2025.
- mpdfdf. Add SM90 MegaMoE by cooperative [EB/OL]. 2026[2026-07-20]. <https://github.com/deepseek-ai/DeepGEMM/commit/f98330768857d65025ca457bf2e9ca168f5fd2c5>.
- Muuuchen. pdl_exp: Programmatic dependent launch experiments [EB/OL]. 2026[2026-07-20]. https://github.com/Muuuchen/pdl_exp/tree/252d629011bc29e62307691609f9bdfcda8337.
- Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating deep neural networks execution with advanced operator fusion [C/OL]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 883-898. DOI: 10.1145/3453483.3454083.
- Nrusimha A, Brandon W, Mishra M, et al. Flashformer: Whole-model kernels for efficient low-batch inference [EB/OL]. 2025. <https://arxiv.org/abs/2505.22758>.
- NVIDIA. CUDA programming guide 13.3: CUDA graphs [EB/OL]. 2026[2026-07-20]. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html>.
- NVIDIA. CUTLASS 4.5.1 documentation: Dependent kernel launches [EB/OL]. 2026[2026-07-20]. https://docs.nvidia.com/cutlass/4.5.1/media/docs/cpp/dependent_kernel_launch.html.
- NVIDIA. CUTLASS example 63: Hopper GEMM with L2 weight prefetch [EB/OL]. 2026[2026-07-20]. <https://github.com/NVIDIA/cutlass>.
- NVIDIA. CUDA programming guide 13.3: Programmatic dependent launch and synchronization [EB/OL]. 2026[2026-07-20]. <https://docs.nvidia.com/cuda/cuda-programming-guide/>.
- RayWang96. Draft: Add green-context split-kernel MegaMoE features [EB/OL]. 2026[2026-07-20]. <https://github.com/deepseek-ai/DeepGEMM/commit/bb837421bd9480224107f4de60cecdca99f05e01>.
- Shah J, Bikshandi G, Zhang Y, et al. Flashattention-3: Fast and accurate attention with asynchrony and low-precision [C/OL]//Globerson A, Mackey L, Belgrave D, et al. Advances in Neural Information Processing Systems: volume 37. Curran Associates, Inc., 2024: 68658-68685. DOI: 10.52202/079017-2193.
- Shi Y, Yang Z, Xue J, et al. Welder: Scheduling deep learning memory access via tile-graph [C/OL]//

17th USENIX Symposium on Operating Systems Design and Implementation. 2023: 701-718.

<https://www.usenix.org/conference/osdi23/presentation/shi>.